# CityQuest - COMPSCI 732: Individual Report

Romili Townsend
University of Auckland
rtow454@aucklanduni.ac.nz

**Abstract—CityQuest is a GPS-based mobile exploration game designed to encourage physical activity and improve users' familiarity with urban environments through navigation-based tasks. The system assigns challenges at locations within the city that users must travel to in order to complete. Gamification elements such as streaks, levels, badges, and leaderboards are employed to increase user motivation and engagement. This report outlines the motivation, design, and implementation of this system developed for the COMPSCI 732 course, with a particular focus on methodology and tooling, and compares it to existing solutions.**

## 1 Introduction

Less than half of the New Zealand population meets recommended physical activity guidelines [**?**]. This highlights the need for interventions that encourage greater participation in physical activity and increase user engagement.

A systematic review and meta-analysis found that gamification interventions are effective at promoting physical activity across diverse age groups and genders and may outperform alternative interventions [**?**]. Gamification therefore presents a promising approach for encouraging physical activity while maintaining user motivation and engagement.

Beyond physical activity, environmental interaction how people learn to navigate their surroundings. Environmental familiarity has been associated with improved wayfinding performance, including reduced navigation errors and shorter route completion times [**?**]. In addition, repeated exposure to an environment has been shown to support the acquisition of spatial knowledge in multiple dimensions, including landmark recognition and route learning [**?**]. Together, these findings suggest that increasing familiarity with local environments may improve users' navigation ability through the development of stronger spatial knowledge.

This project addresses both objectives by employing gamified, navigation-based activities that promote exploration and interaction with local environments, with the aim of increasing environmental familiarity and physical activity levels.

## 2 Goals

Create a GPS-based city exploration game that rewards movement and exploration of the local environment.

- Encourage physical activity through gameplay that requires movement, reinforced by reward-based incentives.
- Promote environmental familiarity through interaction with local landmarks and points of interest.
- Promote an engaging, gamified alternative to traditional navigation tools.

## 3 Related Work

Several existing solutions incorporate location-based interaction and exploration-based gameplay. Notable examples include *Pokemon Go* and *Ingress*, both of which use GPS-backed movement as a core gameplay mechanic to encourage real-world exploration and physical activity.

While these location-based games require users to physically travel to geographic locations, interaction with these environments is primarily mediated through game mechanics rather than direct engagement with the world. In these systems, players interact with virtual elements overlaid on geographic locations, such as collecting items or capturing creatures.

In contrast, this project requires users to engage directly with the physical environment through curated, contextually relevant real-world tasks. Unlike existing systems, the primary interaction occurs within the physical environment rather than within the application itself.

In the domain of navigation, Google Maps is widely used application that supports local discovery and route planning. Features such as *Search This Area* allow users to explore points of interest within a geographic region, including categories such as restaurants and parks. Location discovery is further supported through filtering based on various criteria, such as proximity, user ratings, and popularity. The platform also provides optimised route planning through turn-by-turn navigation instructions, prioritising minimising travel time.

While these features support navigation and location discovery, Google Maps is designed primarily to optimise travel efficiency by identifying the fastest route to a location rather than encourage physical activity and exploration. In Google Maps, users typically specify a destination first and are subsequently guided there.

In contrast, CityQuest provides users with daily, location-backed challenges that encourage exploration, environmental interaction, and physical activity. In addition, CityQuest incorporates game mechanics to encourage exploration and movement, whereas Google Maps does not.

## 4 Design: Software architecture (e.g. class diagram)? User interface (e.g. screen diagram)? Why this design?

We created an ERD with PlantUML during the initial system planning phase to visualise the relationships between entities within the system. As the system evolved, we updated this ERD to reflect the changes in the system. This final ERD can be see in Figure **??**.

As can be seen in this ERD, there is an inherent relationship between each entity type in the database. For example, users are assigned user challenges, each of which corresponds to a challenge, and each of which which has an associated category. This highly related nature of the entities in our data influenced our decision in choosing a relational database such as PostgreSQL over a document-oriented database such as MongoDB.

Low-fidelity Figma mockups were created for the Challenge list, challenge map, achievements, and profile pages of the frontend system user interface, as can be see in Figures **??** and **??**. These prototypes were created to serve as inspiration for the in-development

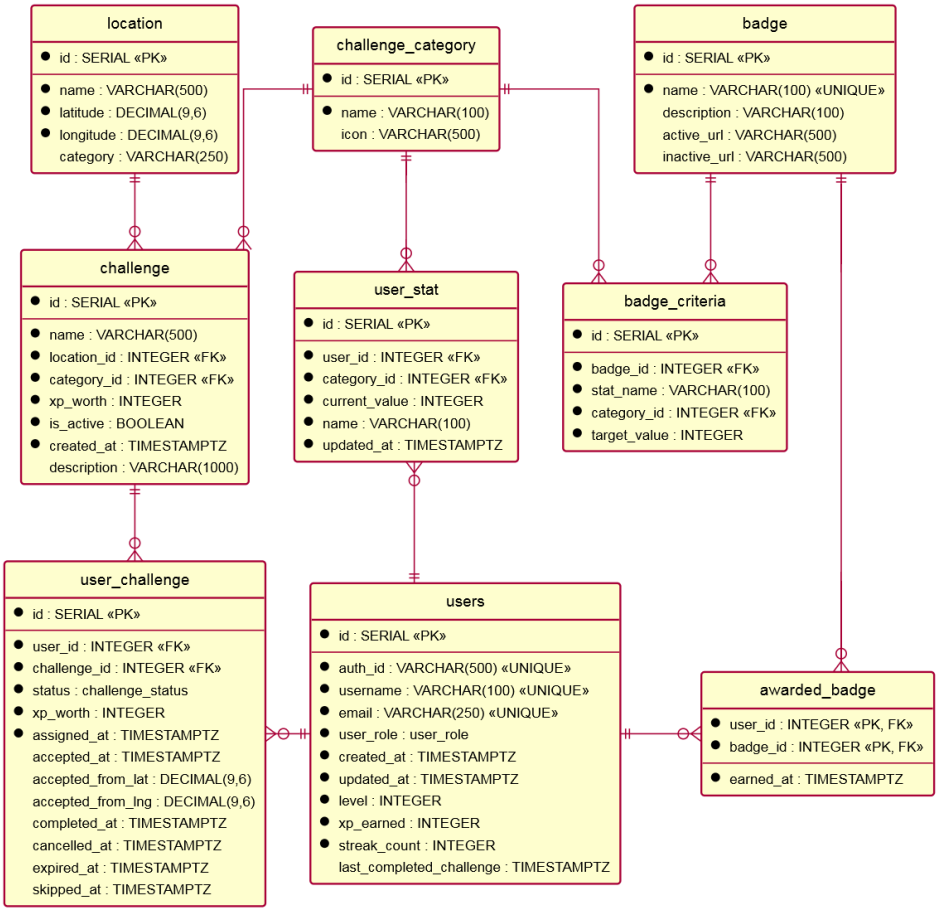


Figure 1: Entity Relationship Diagram of the CityQuest project

system, identify early and UX concerns, and ensure all group members had a consistent vision for the system.

## 5 Implementation: What have you implemented?

## 6 Testing: How have you tested?

Backend testing was performed using Vitest. Unit tests were used to verify the functionality of individual components, while integration tests validated the database interactions, schema constraints, and trigger functionality. Supertest was used to test API endpoints by simulating HTTP requests and verifying that the correct response status codes and response bodies were returned. Additionally, TypeScript type checking was used to detect static typing errors during development.

Frontend component testing was performed using Vitest and React Testing Library. These tests verified component rendering, user interactions, and application state changes within a simulated browser environment provided by jsdom.

End-to-end testing was performed using Playwright. These tests validated user workflows by simulating user interactions with the application through a web browser, ensuring that the frontend and backend component functioned correctly when integrated into the overall system. The end-to-end test suite was incorporated into the GitHub Actions CI/CD pipeline to automatically execute on every commit, providing continuous verification of the application's functionality.

One limitation of the testing suite is a lack of code test coverage analysis. While the unit, integration, and end-to-end tests provide

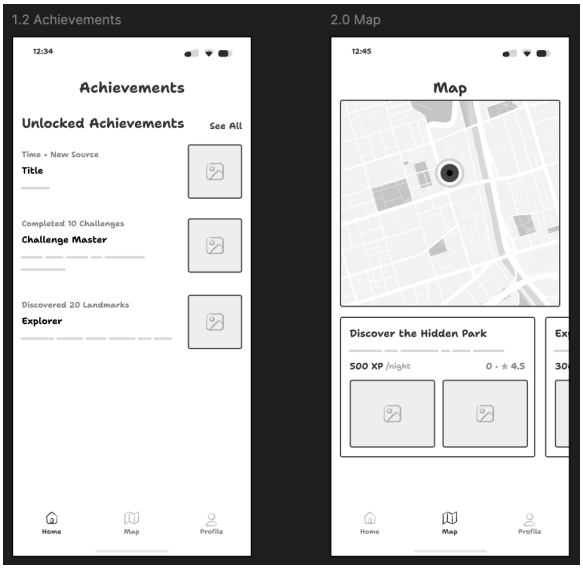


Figure 2: Figma mockups of the Achievements and Map view pages

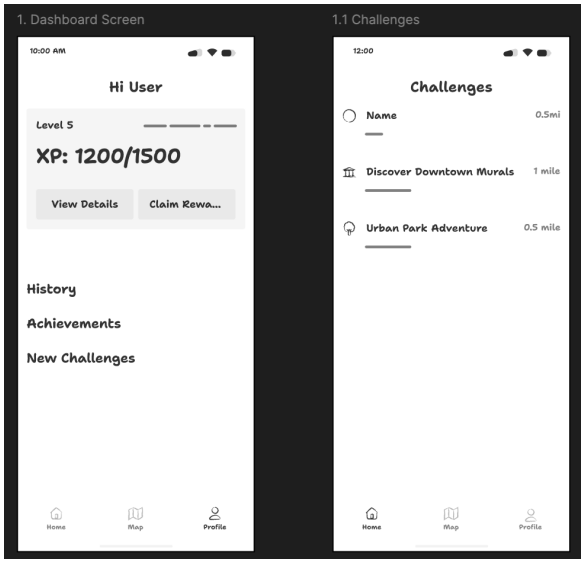


Figure 3: Figma mockups of the Profile and Challenge view pages

functionality assurances across many components of both the backend and frontend codebases, there is no coverage testing set up to measure how much of the source code is executed during testing. As a result, there may be hidden bugs in these areas of the codebase that are not detected from running the test suite. For the purpose of this report, test coverage analysis with Vitest was conducted for the frontend and backend codebases, the results of which are summarised in Table **??**. Full results can be found in the Figures **??** and **??**.

| Module | Statement Coverage | Branch Coverage | Functions Coverage | Line Coverage |
|---|---|---|---|---|
| Frontend | 79.14% | 70.16% | 77.3% | 82.1% |
| Backend | 73.65% | 61.22% | 67.11% | 74.02% |

Table 1: Frontend Test Coverage by Codebase

As can be seen in the Table **??** and Figures **??** and **??**, the frontend achieves higher coverage than the backend in all metrics. This suggests that frontend components are tested more thoroughly by the test suite. The most significant gap missed by the test suite is the Domain Access Objects (DAOs), with a branch coverage of 0%.

## 7 Methodology: How has your team managed the project?

### 7.1 Development

The project adopted an iterative, sprint-based development cycle based on Agile methodologies. Work was organised into weekly sprints. At the beginning of each sprint, tasks were defined, prioritised, and allocated to individual team members. To reduce ambiguity in task completion, each task included clearly defined acceptance criteria specifying what conditions needed to be met for the task to be considered complete.

The team held weekly sprint meetings to coordinate development and check-in on each member's progress. During these meetings, attendance was recorded and discussions were documented in meeting minutes for future reference. If any team member was absent, they were briefed via the team Discord server and could refer to the full meeting minutes for further information. Meeting times were scheduled each week based on the overlapping availability of all team members, collected using the scheduling tool Timeful, to maximise attendance. Due to inconsistent team member availability and differing amounts of discussion required each week, the scheduled duration of meetings varied from week to week, and meetings were flexibly adjusted to accommodate these factors.

Task prioritisation was managed using the MoSCoW method, categorising work into Must have, Should have, Could have, and Won't have (out of current scope) requirements, ordered from highest to lowest priority.

For each task that we created, we added a corresponding issue ticket to GitHub. Commits referenced these issues so that team members could easily trace changes back to specific commits. Tasks were organised and tracked using a Github Projects Kanban board which provided a visual representation of the progress of the project and current sprint. Table **??** outlines the different Kanban board task categories and their status.

| Category | Task Status |
|---|---|
| Product Backlog | Not yet assigned to a sprint |
| To Be Decided | Not yet assigned to a team member |
| Sprint Backlog | Part of the active sprint but not started |
| In Progress | Actively being worked on |
| In Review | Completed pending pull request approval |
| Done | Completed |

Table 2: Kanban board categories

During the "In Review" stage, if a pull request was rejected, then the task would be moved backwards to the "In Progress" status and then later moved forward to "In Review" again upon the requested changes being made.

Changes were made in sub-branches of the *feature* and *bug* branches. Upon a feature being completed, a pull request was made to bring those changes to the main branch, with the related issues being referenced in the pull request message. This was done to ensure the main branch only contained completed, tested features. All pull requests were peer reviewed by at least one other team member before being merged into the main branch, ensuring code quality and consistency.

In addition, a Discord webhook was configured to automatically notify the team when new pull requests were created, updated, or merged. This helped keep everyone informed of ongoing changes and improved visibility of development activity without requiring constant manual checking of GitHub.

### 7.2 Deployment

The frontend was deployed as a Single Page Application (SPA) using Vercel's free tier. Vercel integrates with GitHub to automatically build and deploy the frontend whenever commits are pushed to configured branches, enabling a rapid deployment workflow.

Supabase was used to host the PostgreSQL database and provide authentication services. The API server, which communicates with the database, was deployed as a Render web service using the free tier. Similar to the Vercel frontend deployment, Render integrates with GitHub and automatically builds and deploys the API server whenever commits are pushed to configured branches.

One limitation of this deployment method is that Render's free tier service automatically spins down after 15 minutes of inactivity. When the service has spun down, the first request experiences a cold-start delay and times out while the service restarts. This startup process typically takes approximately one minute before the API becomes responsive to requests again.

## 8 Tooling: What tools did you use to assist the development process (e.g. git, Jira, ChatGPT, etc...)? How effective were those tools? Any drawbacks (especially with the GenAI stuff)?

Generative AI was used by some team members to increase efficiency, particularly for menial tasks where the code is easy to

validate.

Copilot code review was used in multiple instances to check for any bugs, syntax errors, and suggest considerations. However, this was not used as a substitute to human-run peer reviews but rather in addition to; all pull requests to the main branch were peer reviewed by at least one other team member.

## 9 Discussion: Have the goals been achieved? Problems?

A user study could be valuable to investigate whether this city exploration game had an impact on users' familiarity with their surroundings, and whether it increased users' physical activity levels, particularly average daily step counts. Unfortunately, due to the nature of the course, it was not feasible to conduct a user study.

In our project proposal we marked 13 tasks as must haves, 6 tasks as should haves, 10 tasks as could haves, and 4 tasks as out of scope. Of these tasks, all must haves were completed, all should haves, 2 could haves, and no out of scope tasks.

The following table compares, for every prioritisation category, the number of tasks completed vs the number of tasks assigned.

| Prioritisation | Completed tasks | Assigned tasks | Percent completed |
|---|---|---|---|
| Must have | 13 | 13 | 100% |
| Should have | 6 | 6 | 100% |
| Could have | 2 | 10 | 20% |
| Total (within scope) | 22 | 29 | 75.86% |
| Won't have (outside scope) | 0 | 4 | 0% |

Table 3: Task completion summary by prioritisation category

## 10 Conclusion: Conclusions? Lessons learned? Reflections? Future work?

### 10.1 Lessons Learned

### 10.2 Future Work

While all the core functionality of our application was met, the project can be improved to boost the engagement of users and become more accessible.

One area of future work is improving the accessibility of CityQuest for users with disabilities. The current system does not include accessibility settings or provide accessibility information for challenges. CityQuest also assumes that users are able to physically travel to and interact with locations in the environment, which may exclude individuals with mobility impairments.

Future iterations of the system should introduce adjustable movement requirements, allowing users with limited mobility to be assigned challenges of a shorter travel distance. Additionally, the system could incorporate accessible route generation and provide accessibility information for each challenge location to better support participation among a diverse user base.

## 11 Appendix

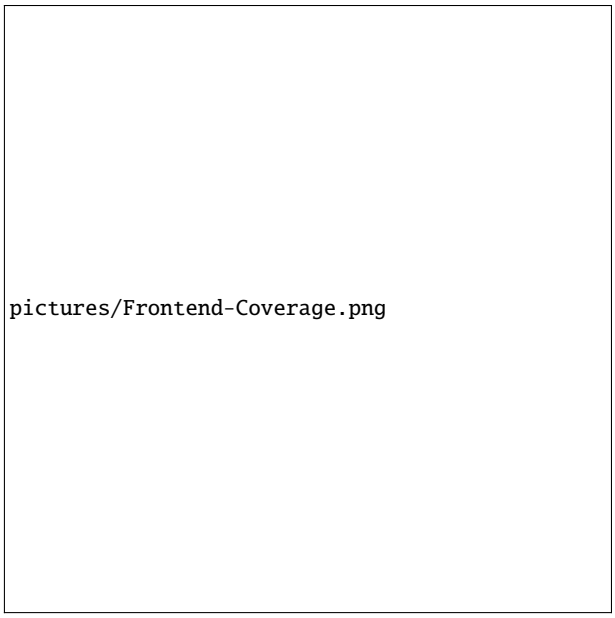


Figure 4: Frontend Coverage Analysis Results

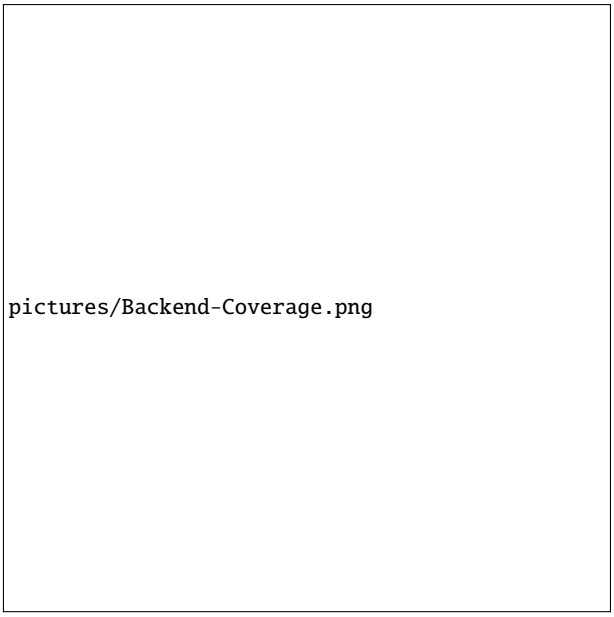


Figure 5: Backend Coverage Analysis Results

## References

[1] Nick Garrett, Philip J. Schluter, and Grant Schofield. Physical activity profiles and perceived environmental determinants in new zealand: A national cross-sectional study. *Journal of Physical Activity and Health*, 9(3):367 – 377, 2012.

[2] A Mazeas, M Duclos, B Pereira, and A Chalabaev. Evaluating the effectiveness of gamification on physical activity: Systematic review and meta-analysis of randomized controlled trials. *Journal of Medical Internet Research*, 24(1):e26779, 2022.

[3] Michael J. O'Neill. Effects of familiarity and plan complexity on wayfinding in simulated buildings. *Journal of Environmental Psychology*, 12(4):319–327, 1992.

[4] Qi Ying, Christopher Hilton, and Sara Irina Fabrikant. Do mobile maps help or hinder? investigating their role in spatial knowledge acquisition across repeated navigation episodes. *Journal of Environmental Psychology*, 111:103031, 2026.